Informe de Laboratorio 03

Tema: Ajax y NodeJS

Nota

Estudiante	Escuela	Asignatura
Francis Willian Ayamamani Añasco fayamamani@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Programación Web 2 Semestre: I Código: GUIA-PRLD-001

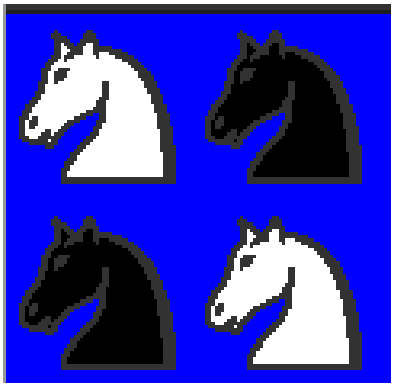
Laboratorio	Tema	Duración
03	Ajax y NodeJS	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	Del 5 Junio 2023	Al 5 Junio 2023

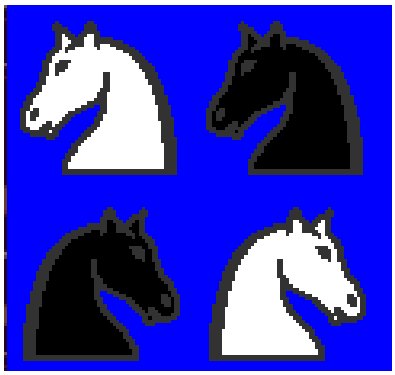
1. Tarea

- Para resolver los siguientes ejercicios sólo está permitido usar ciclos, condicionales, definición de listas por comprensión, sublistas, map, join, (+), lambda, zip, append, pop, range.

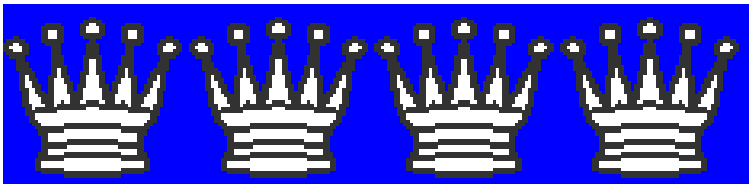
  1. Implemente los métodos de la clase Picture. Se recomienda que implemente la clase picture por etapas, probando realizar los dibujos que se muestran en la siguiente preguntas.
  2. Usando únicamente los métodos de los objetos de la clase Picture dibuje las siguientes figuras (invoque a draw):
     - (a)



- (b)



- (c)



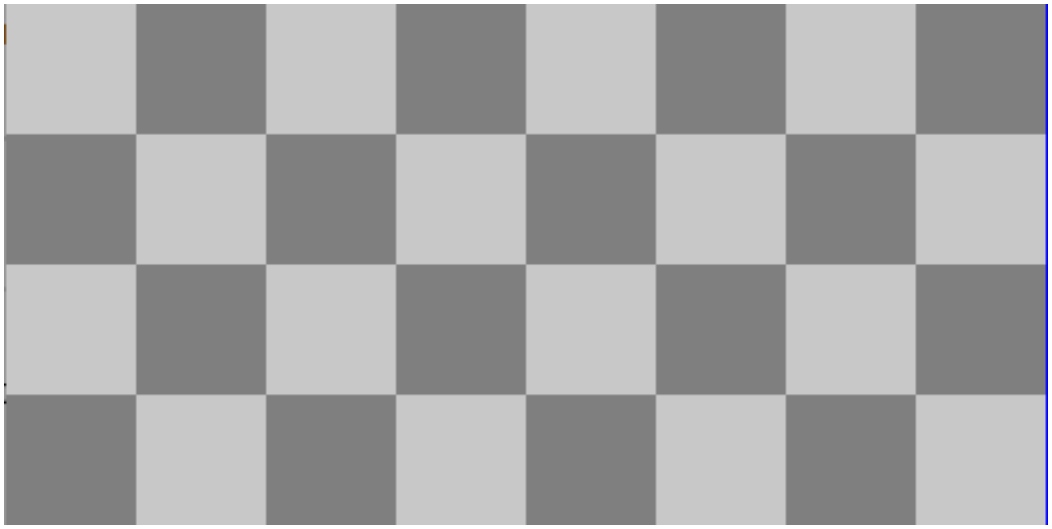
- (d)



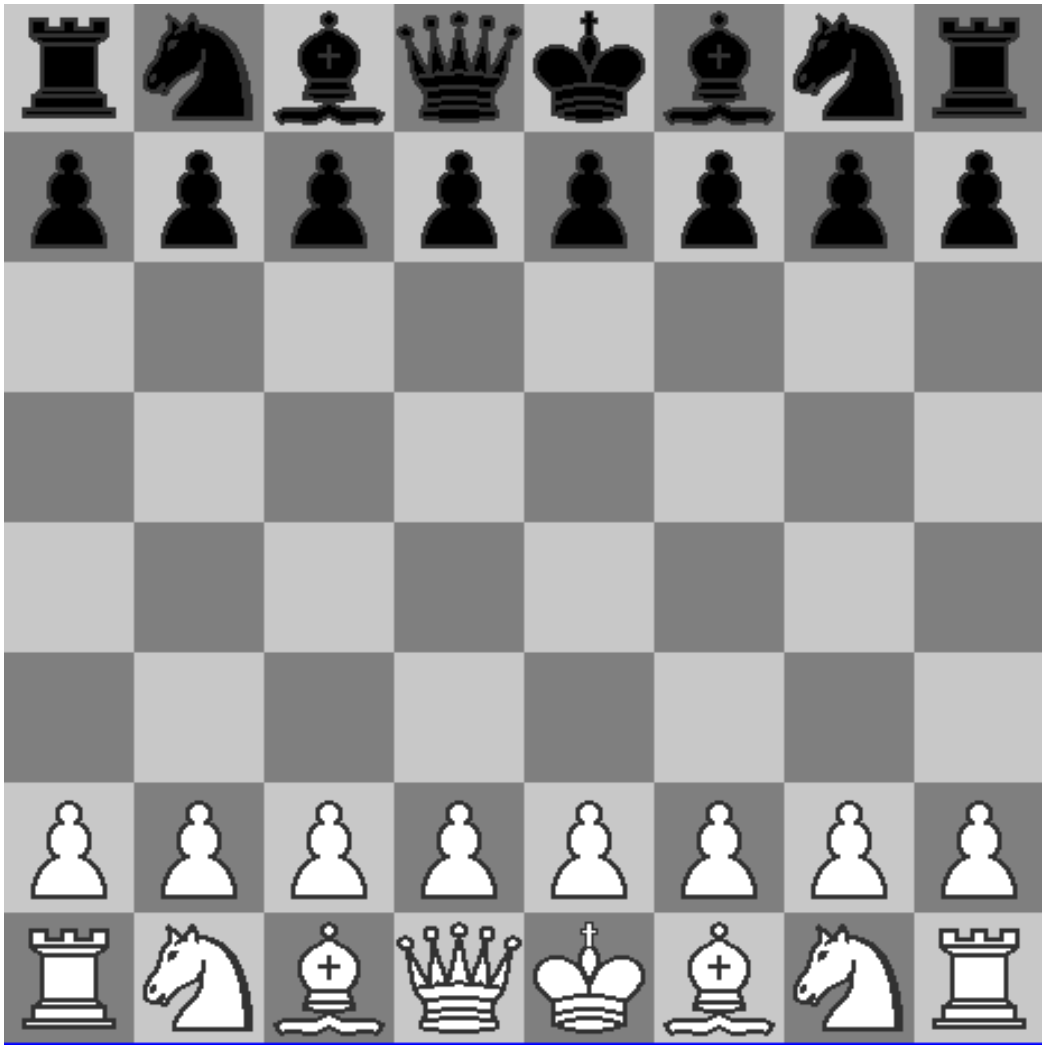
- (e)



- (f)



- (g)



2. MARCO CONCEPTUAL

- Editor Vim
  - Vim es un editor de texto muy eficiente para programar en consola que no tiene nada que envidiar a los editores gráficos.
- Instalar Vim en
  - GNU/Linux

```
apt-get install vim
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://www.vim.org/download.php`
  - MacOS

```
brew install macvim
```

- Git
  - Git es un sistema de control de versiones creado por Linus Torvalds. Es eficiente, confiable.
- Instalar Git
  - GNU/Linux

```
apt-get install git
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://git-scm.com/download/win`
  - MacOS
    - Para descarga en sistemas MacOS `https://git-scm.com/download/mac`

```
brew install git
```

- python
  - Para un instalación local en sistemas operativos MS Windows utilice: https://www.python.org/downloads/w
  - Tambien ud puede probar código en herramientas online: `https://www.programiz.com/python-programming/online-compiler/`
  - Python en Debian GNU/Linux:

Listing 1: Debian Linux viene con Python 3 y Python 2 preinstalados.

```
python3 --version
    Python 3.9.2
```

Listing 2: Instalemos pip, una herramienta que instalará y administrará los paquetes de programación que queramos usar en nuestros proyectos de desarrollo.

```
sudo apt-get install -y python3-pip
```

Listing 3: Paquetes y herramientas de desarrollo más para instalar para garantizar que tengamos una configuración sólida para nuestro entorno de programación:

```
sudo apt-get install build-essential libssl-dev libffi-dev
    python3-dev
```

- Configurar un entorno virtual:
  - Los entornos virtuales permiten tener un espacio aislado en los proyectos Python
  - Garantizando que cada proyecto pueda tener su propio conjunto de dependencias que no interrumpirán a otros proyectos.
  - Manejando diferentes versiones de los paquetes. Esto es especialmente importante cuando se trabaja con paquetes de terceros.
  - Puede varios entornos de programación.
  - Cada entorno es un directorio en la que se ubicaran sus scripts.
  - Usaremos el módulo venv , que es parte de la biblioteca estándar de Python.

Listing 4: Entorno de Linux-Instalemos venv escribiendo:

```
sudo apt install -y python3-venv
```

Listing 5: Entorno de Linux-Para crear un ambiente elija en qué directorio.

```
mkdir -p ~/unsa/pw2/labs/lab04/entorno
    cd ~/unsa/pw2/labs/lab04/entorno
```

Listing 6: Entorno de Linux-En el directorio crea un entorno virtual ejecutando el siguiente comando:

```
virtualenv -p python3 .
```

Listing 7: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
source ../bin/activate
```

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- `https://github.com/rescobedoq/pw2/tree/main/labs/lab04/Tarea-del-Ajedrez`
- URL para el laboratorio 04 en el Repositorio GitHub.
- `https://github.com/rescobedoq/pw2/tree/main/labs/lab04`

4. Resolucion de la tarea

4.1. Explicacion del Codigo

Listing 8: Clase Picture

```
from colors import *
class Picture:
  def __init__(self, img):
    self.img = img;
```

```
def __eq__(self, other):
  return self.img == other.img

def _invColor(self, color):
  if color not in inverter:
    return color
  return inverter[color]

def verticalMirror(self):
  """ Devuelve el espejo vertical de la imagen """
  vertical = []
  for value in self.img:
     vertical.append(value[::-1])
  return vertical

def horizontalMirror(self):
  """ Devuelve el espejo horizontal de la imagen """
  horizontal=self.img[::-1]
  return Picture(horizontal)

def negative(self):
  """ Devuelve un negativo de la imagen """
  neg=[]
  for linea in range(len(self.img)):
      negativo=""
      for pixel in range(len(self.img[linea])):
          negativo+=self._invColor(self.img[linea][pixel])
      neg.append(negativo)
  return Picture(neg)

def join(self, p):
  """ Devuelve una nueva figura poniendo la figura del argumento
      al lado derecho de la figura actual """
  nuevo=[]
  for linea in range(len(p.img)):
      nuevo.append(self.img[linea]+p.img[linea])
  return Picture(nuevo)

def up(self, p):
  nuevo=p.img+self.img
  return Picture(nuevo)

def under(self, p):
  """ Devuelve una nueva figura poniendo la figura p sobre la
      figura actual """
  new=[]
  for (linea1,linea2) in zip(range(len(p.img)),range(len(self.img))):
      lineas=""
      for (pixel1,pixel2) in zip(p.img[linea1],self.img[linea2]):
          if pixel1==" ":
              lineas+=pixel2
          else:
              lineas+=pixel1
      new.append(lineas)
  return Picture(new)
```

```
    def horizontalRepeat(self, n):
      """ Devuelve una nueva figura repitiendo la figura actual al costado
          la cantidad de veces que indique el valor de n """
      repeticion=[]
      for lineas in range(len(self.img)):
          repeticion.append(self.img[lineas]*int(n))
      return Picture(repeticion)

    def verticalRepeat(self, n):
      repeticion=[]
      repeticion+=self.img*int(n)
      return Picture(repeticion)
```

- la funcion **verticalMirror**; comienza creando una variable Lista (**vertical**), para luego con una estructura de dato repetitiva (for), recorre cada linea de objeto self.img; que es nuestro elemento local; para luego cambiar el orden de los elementos de esta linea [primero = final]. Finalmente agrega esta linea a la Lista antes creada, mediante el metodo **append()**.
- la funcion **horizontalMirror**; comienza creando una variable Lista (**horizontal**), para luego cambiar el orden de las lineas de **self.img** [primero = final]. Finalmente se le asigna estos cambios de la variable .img a horizontal.
- la funcion **negative**; comienza creando una variable Lista (**neg**), para luego recorrer las lineas de la ficha, luego de ello dentro de este recorrido se crea una variable local, que guarde la nueva linea negativa. Y para ello, hacemos un for anidado, que recorra cada pixel o en este caso cada elemento de la cadena, para luego cambiarla por su negativo, segun el diccionario.
- la funcion **join**; comienza creando una variable Lista (**nuevo**), para luego concatenar los contenidos de cada linea i de **self.img** y **p.img**; mediante el bucle for.
- la funcion **under**; comienza creando una variable Lista (**new**), para luego cambiar los pixeles (caracteres de la cadena de texto), mediante una estructura if que verifica si el pixel esta vacio, entonces lo remplaza con el pixel de self.img; pero si no entonces introduce el pixel de p.img
- la funcion **up**; comienza creando una variable Lista (**nuevo**), para luego concatenar las listas:

Listing 9: Codigo de ejemplo

```
lista1=[1,2,3]
lista2=[5,6,7]
lista3= lista1+lista2
print (lista3)
```

Listing 10: Resultado de Consola

```
[1,2,3,4,5,6,7]
```

- la funcion **horizontalRepeat**; utiliza la misma logica que el algoritmo de **horizontalMirror**, agregando el simbolo ***** para poder repetir una linea:

Listing 11: Codigo de ejemplo

```
comentario="Hola pepe"
```

```
        print (comentario*int(2))
```

Listing 12: Resultado de Consola

```
        Hola pepeHola pepe
```

- la funcion **verticalRepeat**; al igual que **horizontalRepeat**, utiliza la simbolo de ***

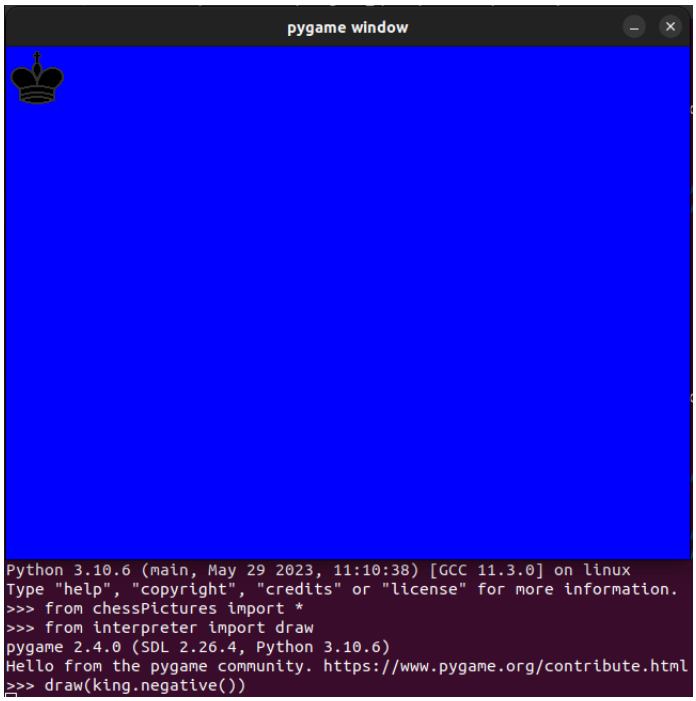
5. Commit-Estructura

5.1. Commit-Laboratorio 04

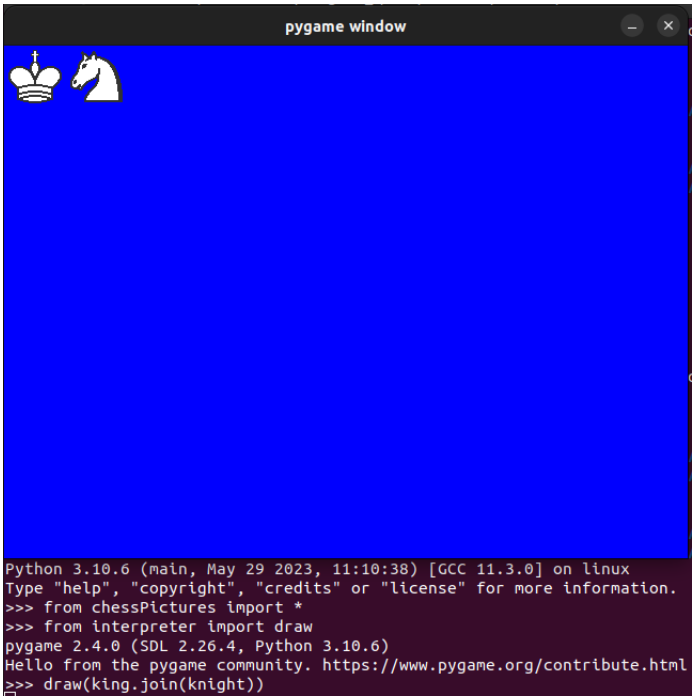
5.1.1. Lista en GITHUB



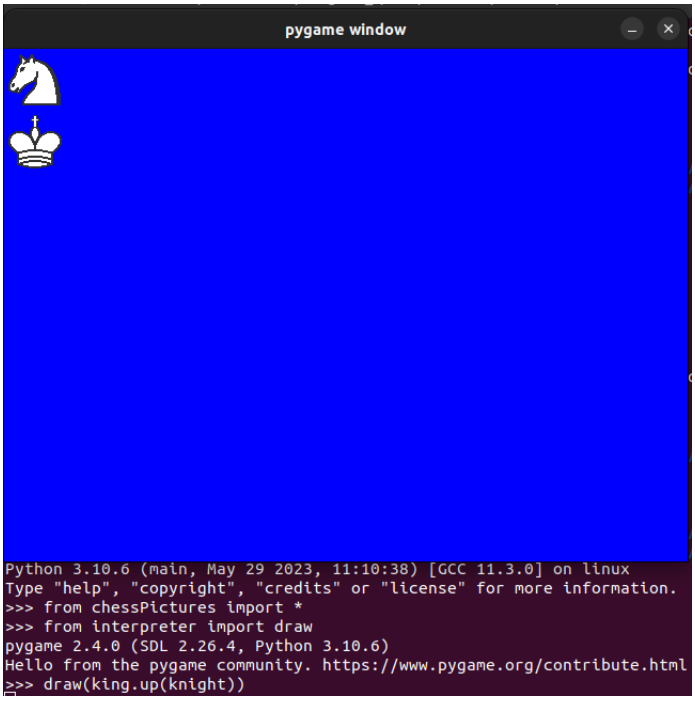


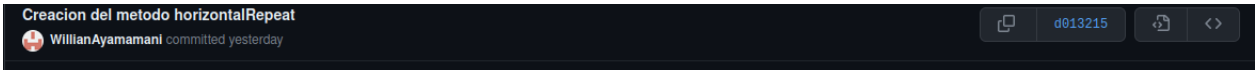





pygame window
Python 3.10.6 (main, May 29 2023, 11:10:38) [GCC 11.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from chessPictures import *
>>> from interpreter import draw
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
>>> draw(king.join(knight))

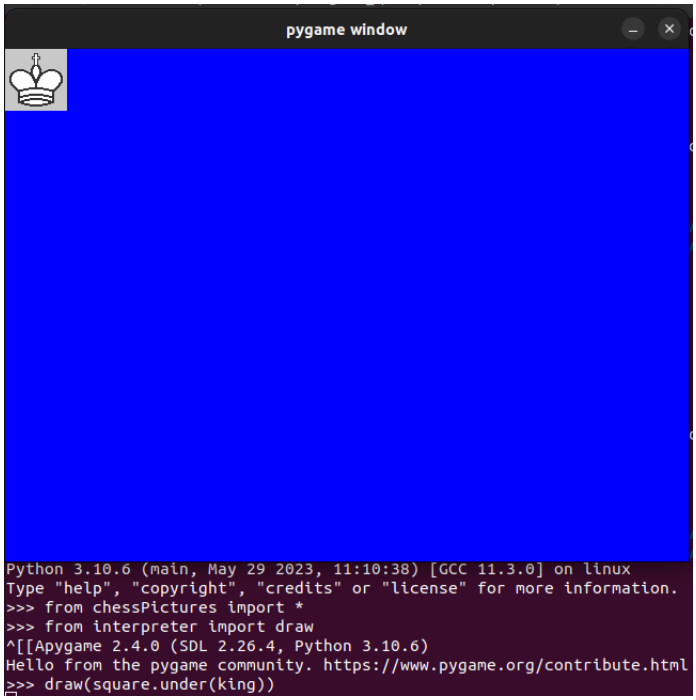

Creacion del metodo up
WillianAyamamani committed yesterday
75b18d5

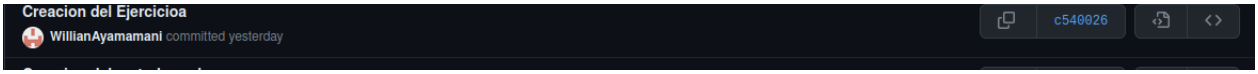

pygame window
Python 3.10.6 (main, May 29 2023, 11:10:38) [GCC 11.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from chessPictures import *
>>> from interpreter import draw
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
>>> draw(king.up(knight))

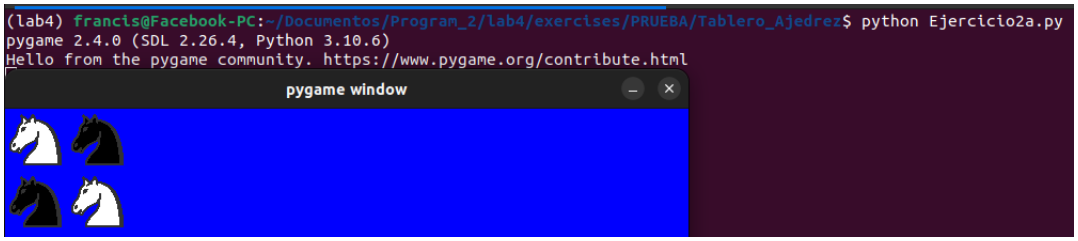

Creacion del metodo horizontalRepeat
WillianAyamamani committed yesterday
d013215


pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
>>> draw(king.horizontalRepeat(2))
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
File "/home/francis/Documentos/Program_2/lab4/exercises/PRUEBA/Tablero
pygame.display.update()
pygame.error: video system not initialized
>>> draw(king.horizontalRepeat(2))

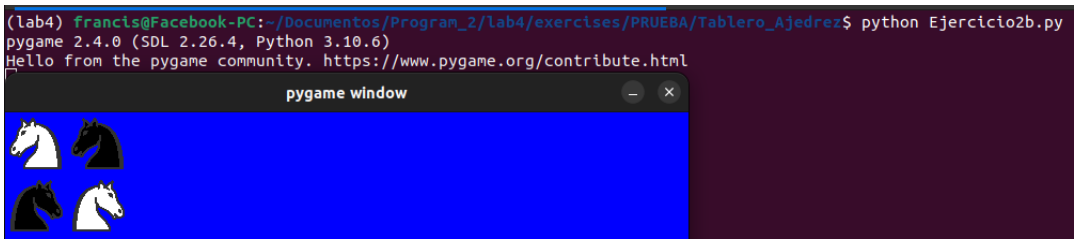

Creacion del metodo under
WillianAyamamani committed yesterday
424df88


pygame window
Python 3.10.6 (main, May 29 2023, 11:10:38) [GCC 11.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from chessPictures import *
>>> from interpreter import draw
^[[Apygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
>>> draw(square.under(king))

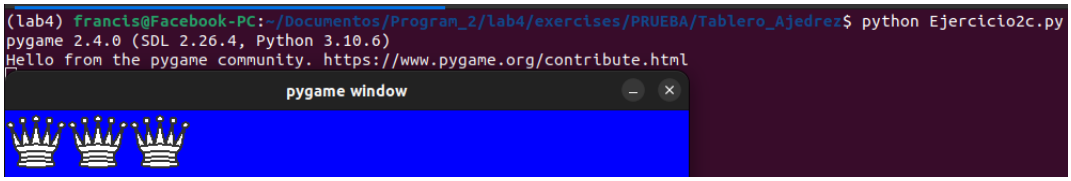

Creacion del Ejercicioa
WillianAyamamani committed yesterday
c540026


(lab4) francis@Facebook-PC:~/Documentos/Program_2/lab4/exercises/PRUEBA/Tablero_Ajedrez$ python Ejercicio2a.py
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
pygame window

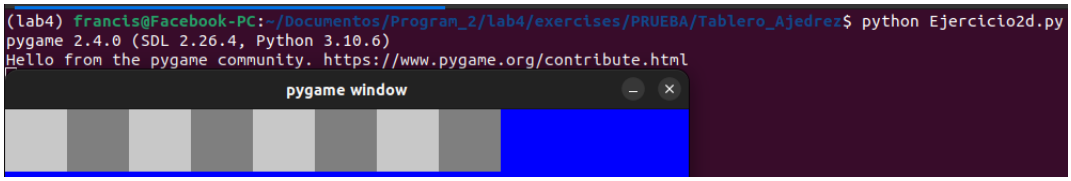

Creacion del Ejerciciob con cambio en picture.py debido a que solo da...
WillianAyamamani committed yesterday
927711a


(lab4) francis@Facebook-PC:~/Documentos/Program_2/lab4/exercises/PRUEBA/Tablero_Ajedrez$ python Ejercicio2b.py
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
pygame window

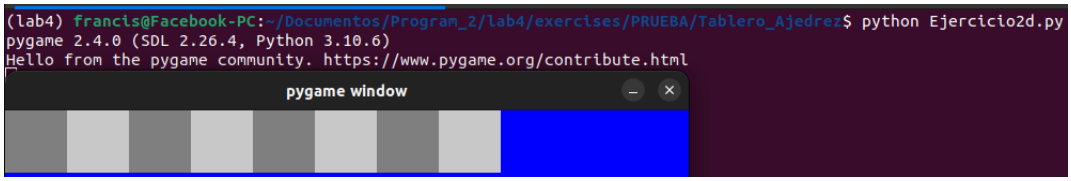

Creacion del EjercicioC
WillianAyamamani committed yesterday
845e9e1

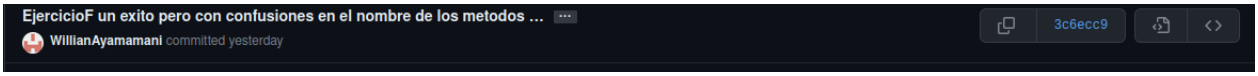

(lab4) francis@Facebook-PC:~/Documentos/Program_2/lab4/exercises/PRUEBA/Tablero_Ajedrez$ python Ejercicio2c.py
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
pygame window


Creando el ejercicioD
WillianAyamamani committed yesterday
ad1e358


(lab4) francis@Facebook-PC:~/Documentos/Program_2/lab4/exercises/PRUEBA/Tablero_Ajedrez$ python Ejercicio2d.py
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
pygame window


Creando el ejercicioE, debido a que me confundi de ejercicio y los re...
WillianAyamamani committed yesterday
290c61a


(lab4) francis@Facebook-PC:~/Documentos/Program_2/lab4/exercises/PRUEBA/Tablero_Ajedrez$ python Ejercicio2d.py
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
pygame window


EjercicioF un exito pero con confusiones en el nombre de los metodos ...
WillianAyamamani committed yesterday
3c6ecc9


(lab4) francis@Facebook-PC:~/Documentos/Program_2/lab4/exercises/PRUEBA/Tablero_Ajedrez$ python Ejercicio2f.py
pygame 2.4.0 (SDL 2.26.4, Python 3.10.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
pygame window


EjercicioG casi un exito recuerda simplificar el codigo
WillianAyamamani committed yesterday
92e443a



5.2. Estructura de laboratorio 04

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab4/
|--- exercises/
    |   |--- esEscalar.py
    |   |--- esUnitaria.py
    |   |--- prueba.py
    |   |--- PRUEBA
            |--- Tablero_Ajedrez
            |   |--- .gitignore
            |   |--- Ejercicio2a.py
            |   |--- Ejercicio2b.py
            |   |--- Ejercicio2c.py
            |   |--- Ejercicio2d.py
            |   |--- Ejercicio2f.py
            |   |--- Ejercicio2e.py
            |   |--- Ejercicio2f.py
            |   |--- Ejercicio2g.py
            |   |--- chessPictures.py
            |   |--- colors.py
            |   |--- interpreter.py
            |   |--- picture.py
            |   |--- pieces.py
            |--- chessMain.py
            |--- chessEngi.py
            |--- chessPicture
            |   |--- nA.png
            |   |--- nC.png
            |   |--- nk.png
            |   |--- nP.png
            |   |--- nR.png
            |   |--- nT.png
```

```
|   |--- piotr-makowski-27LH_0jXKYI-unsplash.jpg
|   |--- wA.png
|   |--- wC.png
|   |--- wk.png
|   |--- wP.png
|   |--- wR.png
|   |--- wT.png
```

6. Cuestionario

- ¿Qué son los archivos *.pyc?
  Los archivos .pyc son archivos de código fuente compilado en el lenguaje de programación Python. Cuando se ejecuta un programa Python, el intérprete de Python compila el código fuente a un código de bytes intermedio llamado "bytecode". Estos archivos .pyc contienen el bytecode compilado, que es una representación más eficiente y rápida del código fuente original.
- ¿Para qué sirve el directorio pycache?
  Cuando se ejecuta un programa Python, el intérprete busca un archivo .pyc correspondiente al archivo .py que se está ejecutando. Si no encuentra el archivo .pyc o si el archivo .pyc existente está desactualizado, el intérprete compila el código fuente y genera un nuevo archivo .pyc. Este archivo .pyc se guarda en el directorio pycache junto con una estructura de carpetas que refleja la ubicación relativa del archivo .py original.
- ¿Cuáles son los usos y lo que representa el subguión en Python?
  En Python, el subguión se utiliza a menudo en la importación parcial de módulos o paquetes,a veces, ademas se utiliza como una variable especial que representa el resultado del cálculo anterior. Esto permite acceder al resultado anterior sin necesidad de asignarlo a una variable explícitamente.

7. Rúbricas

7.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

7.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	X	2	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	1	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	1	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	1	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	X	1	
Total		20		12	

8. Referencias

- https://www.w3schools.com/java/default.asp
- https://www.geeksforgeeks.org/insertion-sort/
- https://github.com/WillianAyamamani/pw2-lab-d-23a/tree/main/lab1